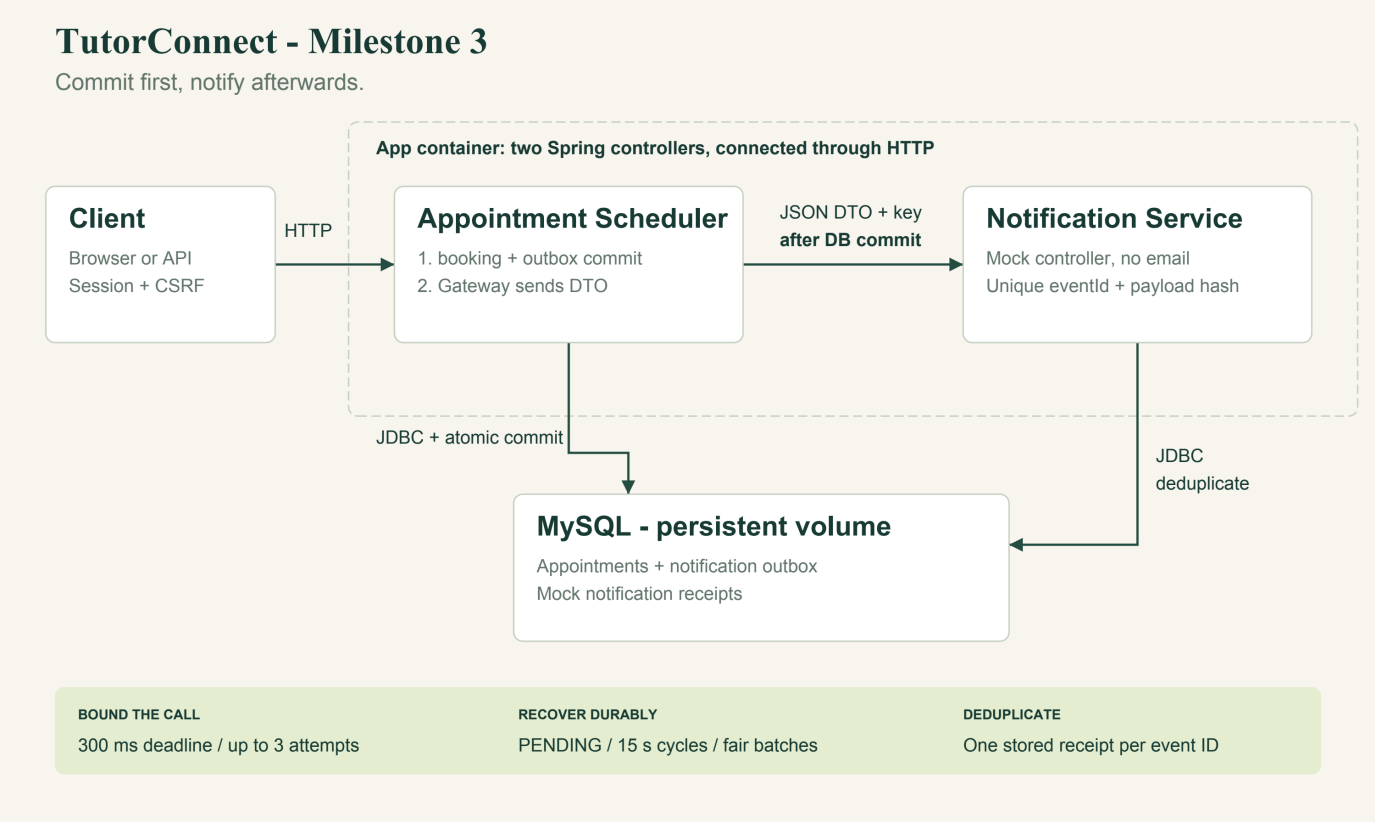
CMPE 172 | MILESTONE 3

1. Distribution boundary and interface

TutorConnect | Yinqi Chen | October 18, 2026

TutorConnect Milestone 3 extends the existing JDBC appointment scheduler with a mock notification boundary, fault handling, observability, Docker packaging and CI. Customers continue to book and cancel through the original session-authenticated interface.



The required Client -> Appointment Scheduler -> Notification Service boundary is an actual HTTP call through NotificationGateway, not a direct Java method call. The mock is a second Spring controller in the same app container, as permitted by the assignment. The dotted boundary in the diagram makes this deployment choice explicit; it does not imply a separate notification container.

One coarse-grained request

```
public record NotificationRequest(String eventId,
    String recipient, String type, String message) {}
```

The DTO contains everything the mock needs to record a notification. It avoids separate remote calls for customer details, appointment details and message fragments. The scheduler owns appointment rules; the notification interface owns acceptance and deduplication. BOOKING_CONFIRMED and CANCELLATION_CONFIRMED share the same interface. A future reminder channel can reuse this boundary with an added supported message type.

2. Commit-then-notify and durable fallback

BookingTransaction inserts the appointment and its notification_outbox event in the same READ_COMMITTED transaction. Cancellation updates the appointment and inserts a distinct cancellation event in its own transaction. If either transaction fails, its business change and event both roll back.

```
// BookingTransaction.book: inside @Transactional
long id = appointments.insert(user, slotId);
notifications.enqueue(user, id, "BOOKING_CONFIRMED");
return id;
```

AppointmentService.book/cancel use NOT_SUPPORTED to suspend any caller transaction. The facade calls the separate transactional Spring bean, waits for it to return committed, then dispatches the event. This is a proxy-based commit boundary: remote I/O is not included in the SQL transaction. NotificationGateway also checks that no actual transaction is active before attempting HTTP.

```
// AppointmentService.book: transaction already committed
long id = retry(() -> transactions.book(user, slot, service));
metrics.booked();
notifications.afterCommit(id, "BOOKING_CONFIRMED");
return id;
```

The outbox row is the fallback. A crash between commit and the first network attempt leaves a PENDING row for recovery. A network failure does not roll back the already committed booking or turn an otherwise successful booking into a notification error. Unexpected dispatch exceptions are logged and leave the persisted event available for another cycle.

Recovery behavior

Every 15 seconds, NotificationRecovery selects at most 20 due PENDING events. Eligible rows are ordered by next_attempt_at, created_at and event_id. Failed retry cycles move their next attempt into the future, so long-failing older records cannot monopolize the first batch forever. A 21-record regression test reproduces and protects this fairness property.

Successful delivery marks the event SENT. Transient failure leaves it PENDING with updated attempts, error category and next attempt time. A permanent rejection marks it FAILED, logs ERROR and requires operator review. After fixing the cause, an operator can reset a FAILED row to PENDING; stable event IDs make replays safe for this mock.

3. Timeouts, retries and idempotency

Bounded remote calls

Defaults are a 200 ms connection timeout, a 300 ms complete-response deadline, up to three attempts and 25 ms between attempts. The gateway uses Java HttpClient sendAsync and waits on the complete response future with a deadline. It cancels timed-out futures rather than waiting indefinitely for a response body. The first delivery cycle therefore has a bounded configured wait; database operations and scheduling add separate overhead.

```
pending = http.sendAsync(request,
    HttpResponse.BodyHandlers.discarding());
int status = pending.get(
    settings.requestTimeout().toMillis(),
    TimeUnit.MILLISECONDS).statusCode();
// On TimeoutException: pending.cancel(true).
```

Connection failures, timeouts, HTTP 408, HTTP 429 and 5xx responses are retryable. Other non-success statuses become permanent failures. Redirects are not followed, preventing service credentials from being forwarded to a redirected host. Interrupted requests preserve the interrupt flag and stop their retry loop. Database lock retries from Milestone 2 remain separate from notification retries.

Idempotent attempts and durable acceptance

The event ID is derived from appointment ID and notification type. The DTO is serialized once per delivery cycle; retries reuse the same ID and payload. notification_receipts has event_id as its primary key and stores a SHA-256 hash of the normalized DTO. First acceptance returns 201. An exact replay returns 200 without another stored receipt. Reusing the key with changed content returns 409.

```
CREATE TABLE IF NOT EXISTS notification_receipts (
  event_id VARCHAR(120) PRIMARY KEY,
  payload_hash VARCHAR(64) NOT NULL,
  recipient VARCHAR(255) NOT NULL,
  type VARCHAR(40) NOT NULL, message VARCHAR(1000) NOT NULL,
  received_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Delivery is at-least-once at the HTTP boundary, with one stored side effect per event in this mock. A receiver may accept a request even when the client loses its acknowledgment; a later replay is therefore expected. This is not a claim of exactly-once delivery to an arbitrary external email system. Different book/cancel events may arrive out of order during an outage; strict cross-event ordering is outside this milestone implementation.

4. Logging, health and metrics

Logging plan and emitted examples

Level	Example emitted event	Purpose
INFO	booking_committed appointmentId=7 slotId=1	Committed business outcome
INFO	notification_accepted eventId=... type=BOOKING_CONFIRMED	First mock acceptance
WARN	notification_attempt_failed eventId=... attempt=1 reason=TIMEOUT	Retryable remote failure
WARN	notification_deferred eventId=... action=scheduled_retry	Persisted fallback
ERROR	notification_rejected eventId=... reason=HTTP_400 action=manual_review	Permanent rejection
ERROR	health_database_down sqlState=08001	Database health failure

The new notification logs use identifiers, type, attempt count and fixed failure categories. They do not log recipient addresses, notification messages, passwords or the service token. Runtime errors are handled server-side; ordinary client responses do not expose stack traces.

Database-aware health endpoint

Public GET /health obtains a pooled connection, runs SELECT 1 with a one-second statement timeout and returns {"status":"UP","database":"UP"} with HTTP 200. SQLException returns {"status":"DOWN","database":"DOWN"} with HTTP 503 and no connection details. The configured Hikari connection acquisition timeout is one second. The Docker image uses this endpoint through a small Java health probe.

Provider-only metric endpoint

GET /api/provider/metrics requires a PROVIDER session. Micrometer records successful/failed bookings, cancellations, delivered notifications and a booking latency timer. The DTO also queries persistent PENDING/FAILED outbox counts. Average booking latency measures the service call, including its initial notification cycle. Metric names and tags have bounded cardinality; user or appointment IDs are never metric labels.

```
{
  "bookingsSucceeded": 1.0, "bookingsFailed": 0.0,
  "cancellations": 1.0, "notificationsDelivered": 2.0,
  "notificationPending": 0, "notificationFailed": 0,
  "bookingAverageMillis": 58.4
}
```

The metric example above is rounded from a real container smoke run. Micrometer counters/timers reset on JVM restart; persisted queue rows and receiver receipts survive application restarts. No external monitoring backend is required for this milestone.

5. Docker deployment and CI

Dockerfile uses a Maven/Temurin Java 17 build stage and a Temurin Java 17 JRE runtime stage. Only the executable JAR and compiled health probe enter the runtime. The container runs as non-root UID/GID 10001. Its health check polls /health. The .dockerignore excludes Git metadata, build output and unrelated artifacts.

docker-compose.yml runs app plus MySQL 8.4. The app waits for the database TCP health check. The project and volume names match the prior Compose configuration so an existing Milestone 2 volume can be reused. Existing bookings are not dropped. NotificationSchemaInitializer applies CREATE TABLE IF NOT EXISTS statements for the two new tables at app startup; a container-init mount alone would not initialize an already-existing volume.

```
docker compose up -d --build --wait
curl -s http://localhost:8080/health
python3 scripts/smoke.py --base-url http://localhost:8080
```

Notification calls target the mock controller over loopback HTTP inside the app container. NOTIFICATION_URL and NOTIFICATION_SERVICE_TOKEN configure the gateway and shared service key. The internal stateless security chain requires that key, while the customer/provider routes retain Session RBAC and browser CSRF protection. The supplied credentials are local demo configuration; no real messaging channel is configured.

GitHub Actions

The existing workflow runs on push and pull_request. One job runs Maven verify with H2 on Java 17, builds the application image, starts its demo profile and executes the Python HTTP smoke script. A second job provisions MySQL 8.4 and runs the same Maven suite against a dedicated test database. Failure diagnostics include container logs; cleanup removes the ephemeral CI container.

Failure scenario and recovery

If the notification endpoint is unavailable, the appointment still commits and the API returns 201. Bounded attempts fail into PENDING. Once the service returns, the recovery worker reuses the same event and marks it SENT after acceptance. If the application restarts during the outage, MySQL retains the event. A notification outage is not treated as a reason to undo the customer appointment.

A healthy Docker application does not mean every optional downstream notification is currently delivered. /health focuses on database-backed application readiness; queue counts and WARN/ERROR events expose notification degradation. A health-check failure marks a Docker container unhealthy; the restart policy restarts exited processes, not merely unhealthy ones.

6. Tests and validation evidence

Validation date: October 18, 2026 (America/Los_Angeles). The final 52-test suite passed with H2 and MySQL 8.4: 0 failures, 0 errors, 0 skipped in each run. The suite also passed inside the official Java 17 Maven container. The Java 17 Docker image built and app/database containers reached healthy state. The Docker HTTP smoke completed customer login, booking 201, cancellation 200 and two delivered notification events.

Test group	Count	Coverage
Existing Milestone 2 tests	31	Booking/RBAC/concurrency/rollback
Milestone3IntegrationTests	10	Commit order, failures, timeouts, recovery, health, metrics, fairness
NotificationStubTests	5	Service auth, malformed input, replay/conflict, concurrent dedupe
NotificationRecoveryTests	5	DB-down health, recovery, exception fallback, metrics
NotificationDownTests	1	No notification server listening; booking still commits

NotificationDownTests points the gateway to an unopened local TCP port. It logs in over HTTP, books a slot, asserts HTTP 201, verifies one active database appointment, and verifies a PENDING event after three failed attempts. This is a real network failure rather than a mocked successful gateway call.

Other integration tests use an HTTP stub that disconnects, delays its response, returns 503 or rejects with 400. The stub probes the appointment through a different database connection before acknowledging the request, demonstrating committed visibility. Recovery tests bring the stub back and verify stable payloads and PENDING-to-SENT transition. The receiver is separately tested with concurrent identical requests and stores one receipt.

```
mvn test
sh scripts/test-mysql.sh
sh scripts/test-mysql.sh \
  '-Dtest=NotificationDownTests'
sh scripts/test-mysql.sh \
  '-Dtest=Milestone3IntegrationTests'
```

Independent read-only review found one recovery starvation issue: always selecting the oldest 20 rows could block newer events. The regression was observed failing, the query was changed to prioritize next_attempt_at, and review recheck found no remaining important defect. Hosted GitHub Actions results are separate from local verification and are not claimed before publishing the branch.

7. Source map, limits and references

Files to explain in the walkthrough

```
notification/NotificationRequest.java
notification/NotificationGateway.java
notification/NotificationDispatcher.java
notification/NotificationRecovery.java
notification/NotificationStubController.java
repository/NotificationOutboxRepository.java
service/BookingTransaction.java
service/AppointmentService.java
observability/HealthController.java
observability/BookingMetrics.java
security/NotificationSecurityConfig.java
Dockerfile / docker-compose.yml
.github/workflows/verify.yml
```

Java source paths are relative to src/main/java/com/example/tutorconnect. Tests are under src/test/java/com/example/tutorconnect. The Chinese run guide and English speech are in docs/milestone3. A real student-recorded code walkthrough of at least five minutes and its video link remain student deliverables.

Practical limits

The mock stores notifications, not real email/SMS. Book/cancel event arrival order is not guaranteed across outages. Recovery is bounded per cycle but continues retrying transient failures across cycles; permanent failures require review. JVM counters reset on restart. The additive schema initializer assumes permission to create the two new tables. Existing schema versions with incompatible notification tables would need a formal migration beyond this first Milestone 3 schema.

References

Assignment: SJSU CMPE 172, Milestone 3 - Integration, Distribution & Deployment, supplied two-page PDF. Course General Rules were referenced by the assignment but were not included in that attachment.

Spring Framework, Transaction-bound Events: https://docs.spring.io/spring-framework/reference/data-access/transaction/event.html (transaction phase semantics; this implementation uses an explicit proxy-return boundary).

Spring Framework, REST Clients: https://docs.spring.io/spring-framework/reference/integration/rest-clients.html

Spring Boot, Metrics: https://docs.spring.io/spring-boot/reference/actuator/metrics.html

Spring Guides, Building a RESTful Web Service with Spring Boot Actuator: https://spring.io/guides/gs/actuator-service/

Source repository: https://github.com/Lushangtu123/CMPE-172-Milestone ; the local Milestone 3 implementation branch is milestone3-integration-deployment.